



LLM Serving under SLOs: A Survey of Engine and Cluster-Layer Scheduling

Journal:	<i>Computing Surveys</i>
Manuscript ID	CSUR-2026-0314
Paper:	Long Survey Paper
Date Submitted by the Author:	02-Mar-2026
Complete List of Authors:	Kondakindi, Shreyash; UC San Diego Gaur, Pratishtha; UC San Diego, Bellur, Umesh; UC San Diego
Computing Classification Systems:	Software and its engineering, Computer systems organization, Resource management, Scheduling, Real-time systems

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

LLM Serving under SLOs: A Survey of Engine and Cluster-Layer Scheduling

SHREYASH KONDAKINDI, HDSI

UC San Diego, United States

PRATISTHA GAUR, HDSI

UC San Diego, United States

UMESH BELLUR, HDSI

UC San Diego, United States

Large Language Model (LLM) serving differs fundamentally from traditional microservice workloads due to autoregressive generation, variable execution time, and substantial memory demands. These characteristics introduce stochastic latency behavior and tightly coupled compute–memory constraints, complicating resource management in production environments. As LLMs are deployed in interactive and mission-critical applications, infrastructure must simultaneously maintain high hardware utilization and satisfy multi-dimensional Service Level Objectives (SLOs), including Time to First Token (TTFT) and per-token latency. This survey presents a structured analysis of techniques for SLO-aware LLM inference across both single-node execution engines and multi-node cluster orchestration. We introduce a unified taxonomy that categorizes existing approaches by system layer and optimization objective, covering mechanisms such as continuous batching, speculative decoding, memory-aware scheduling, admission control, and deadline-aware routing. By comparing academic proposals and production systems, we highlight trade-offs among throughput, latency, fairness, and resource efficiency, and identify open challenges in predictive modeling, multi-tenant isolation, and cross-layer optimization.

Additional Key Words and Phrases: Large language models, SLO-aware scheduling, LLM inference, resource management, latency–throughput trade-offs

ACM Reference Format:
Shreyash Kondakindi, Pratistha Gaur, and Umesh Bellur. 2026. LLM Serving under SLOs: A Survey of Engine and Cluster-Layer Scheduling. *ACM Comput. Surv.* 1, 1 (March 2026), 20 pages. <https://doi.org/10.1145/nnnnnnn>.

1 Introduction

The deployment of Large Language Models (LLMs) has marked a definitive paradigm shift in data center workloads, transitioning from the deterministic, compute-predictable execution patterns of classical microservices to the stochastic, resource-intensive nature of generative AI. As LLMs become embedded in the critical path of industrial applications—ranging from real-time financial co-pilots and medical diagnostic assistants to high-throughput agents—the metrics for success have fundamentally evolved. While the initial wave of research and industrial optimization focused

Authors’ Contact Information: Shreyash Kondakindi, skondakindi@ucsd.edu, HDSI
UC San Diego, United States; Pratistha Gaur, p1gaur@ucsd.edu, HDSI
UC San Diego, United States; Umesh Bellur, ubellur@ucsd.edu, HDSI
UC San Diego, United States.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7341/2026/3-ART
<https://doi.org/10.1145/nnnnnnn>

almost exclusively on maximizing raw throughput to reduce the cost-per-token, the maturity of the field has necessitated a transition toward Service Level Objectives (SLOs). For modern infrastructure providers, the challenge is no longer merely processing as many requests as possible, but rather doing so while honoring multi-dimensional latency guarantees in a multi-tenant environment.

This transition is driven by the economic reality that in a competitive cloud market, throughput without timing guarantees is insufficient for most high-value use cases. This survey aims to synthesize the emerging field of SLO-aware scheduling, providing a systematic overview of the techniques required to tame the inherent complexities of LLM serving.

1.1 The Technical and Physical Tensions of LLM Serving

The difficulty of meeting SLOs in LLM serving is rooted in a fundamental tension between the physical constraints of GPU hardware and the mathematical requirements of the autoregressive transformer architecture. Unlike traditional deep learning models that process inputs in fixed-size batches with predictable, constant-time latencies, LLMs exhibit a dynamic and stateful execution profile that defies standard scheduling heuristics. This complexity is characterized by three primary systemic challenges.

First, workload uncertainty and autoregressive stochastics form the core of the scheduling dilemma. Because LLMs generate tokens sequentially, the total computational work required for a single request is governed by the model's internal decision to stop—a factor that is mathematically impossible to determine with certainty at the time of admission. This variable-length nature breaks the core assumptions of classic scheduling algorithms such as First-Come-First-Served (FCFS). In a shared cluster, a single whale request that generates thousands of tokens can trap the GPU's memory and compute resources, causing significant queuing delays for subsequent shorter requests. Recent surveys have characterized this struggle as "Taming the Titans" [24], highlighting the difficulty of balancing these massive, unpredictable models against finite, static hardware limits.

Second, the serving process is defined by phase heterogeneity. The inference pipeline is bifurcated into two distinct phases with orthogonal resource demands. The *prefill phase*, which processes the initial input prompt, is highly parallelizable and compute-bound, effectively saturating the GPU's arithmetic logic units (ALUs). Conversely, the *decode phase* generates tokens iteratively, a process that is memory-bandwidth bound as it requires loading billions of parameters and a growing Key-Value (KV) cache for every single token produced. A naïve scheduler that prioritizes prefill to minimize Time to First Token (TTFT) may inadvertently starve the decode iterations of existing requests. This phenomenon leads to stuttering in the output stream and violations of Time-Between-Token Latency (TBT) objectives. This inherent throughput-latency tradeoff is a central theme in recent systems like Sarathi-Serve [1] and SOLA [7], which attempt to balance these phases through more granular, iteration-level execution control.

Third, stateful memory coupling represents the ultimate constraint on system concurrency. The memory footprint of an active request is not static; the KV cache, which stores the attention states necessary for efficient generation, consumes significant GPU memory and grows linearly with every token produced [12, 13]. This creates a stateful coupling where the decision to admit a request is a long-term commitment to provide increasing memory capacity across subsequent iterations. If a scheduler overestimates available capacity, it risks memory fragmentation or out-of-memory (OOM) errors, forcing the system into costly request preemptions. These preemptions represent wasted compute and a total failure of the SLO, as the request must eventually be recomputed or reloaded. Consequently, sophisticated admission control, such as the Past-Future scheduler [6], has become a critical requirement for maintaining SLA guarantees.

1.2 The SLO Landscape: From Throughput to Multi-Dimensional Latency

As LLMs are integrated into a broader spectrum of mission-critical services, latency has fragmented into a multi-dimensional set of SLOs, each corresponding to a specific phase of the inference pipeline. The most critical metric for perceived responsiveness in interactive applications is the **Time to First Token (TTFT)**, which measures the interval between request submission and the arrival of the initial output (first output token). In human-computer interaction, TTFT is the primary driver of system fluency, if the TTFT exceeds specific thresholds, users perceive system lag, leading to reduced engagement. However, minimizing TTFT creates a significant scheduling burden, as a burst of high-TTFT-priority requests can create a convoy effect that stalls the generation iterations of already-active requests.

Beyond the initial response, the perceived quality of a streaming output is governed by Time Per Output Token (TPOT) and Time-Between-Token (TBT) latency. TPOT summarizes the average time taken per token generation whereas TBT is a distribution of latencies between consecutive generated tokens in the output stream. Unlike TTFT, TBT is primarily sensitive to memory bandwidth and scheduling jitter. For accessibility applications, such as real-time screen readers, uniformly slower token generation (high but stable TPOT) is often less damaging than high variance in TBT. Jitter in token delivery breaks the natural cadence of reading, making the output feel mechanical. Protecting TBT requires a heartbeat-aware scheduler that can precisely interleave decode iterations between prefill tasks, as seen in systems like SOLA [7] and SLO-Aware Scheduling [10].

Furthermore, requirements shift dramatically for agentic or batch workloads. For agents performing software engineering tasks or document summarization, the Time to Last Token (TTLT) and the end-to-end duration is often the only metric of concern. In these scenarios, a scheduler may intentionally sacrifice TTFT and TBT to maximize batch sizes and operational intensity. This creates a mixed SLO environment where the scheduler must act as a sophisticated arbiter. As identified by Tempo [23] and SLOs-Serve [3], the inability of standard engines to distinguish between these heterogeneous requirements leads to massive goodput loss, a state where the system is high-utilization but failing to meet the specific deadlines required by the diverse applications it serves.

1.3 A Two-Layer Taxonomy of Scheduling

To address these multi-faceted challenges, the research community has developed a diverse array of scheduling strategies operating at different levels of the system stack.

As physical constraints such as memory necessitate specialized management, we categorize the solution space into a two-layer hierarchy. Figure 1 illustrates how global request management interacts with local hardware execution.

Engine-Layer Scheduling involves the micro-management of the execution loop within a single inference server. This layer is responsible for granular interleaving and memory management. We survey innovations such as *continuous batching* [12, 13], which enables iteration-level flexibility; *speculative execution*, which uses draft models as a scheduling lever to prioritize urgent requests as seen in AdaServe [15] and BanditSpec [9]; and *state-aware admission* [6, 7], which ensures the batch remains feasible throughout its lifecycle.

Cluster-Layer Scheduling addresses the macro-level challenge of distributing heterogeneous requests across a fleet of GPU replicas. At this layer, the focus shifts toward global load balancing, multi-tenant fairness, and goodput optimization. In this survey, multi-tenant refers broadly to resource sharing that may occur at multiple levels, including within the same GPU, across nodes in a cluster, or across services sharing common infrastructure. We examine how cluster routers utilize *deadline-aware routing* (e.g., Tempo [23] and SLOs-Serve [3]) and *credit-based isolation* (e.g.,

Shreyash Kondakindi, Pratistha Gaur, and Umesh Bellur

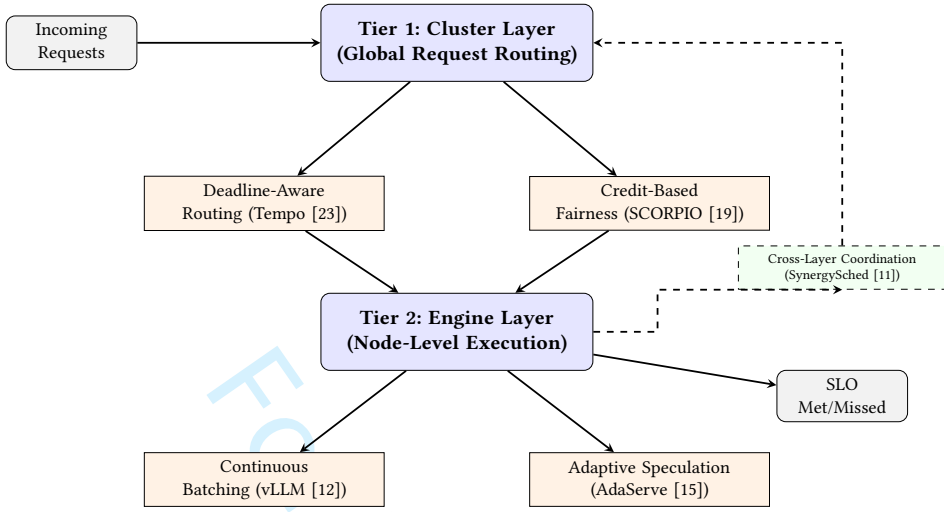


Fig. 1. The hierarchical taxonomy of LLM serving. Tier 1 manages global placement and multi-tenant fairness, while Tier 2 optimizes hardware utilization and fine-grained latency.

SCORPIO [19]) to prevent noisy-neighbor effects. Furthermore, we explore the emerging frontier of **Cross-Layer Co-Design**, where synergistic frameworks like SynergySched [11] share real-time state between the router and engine to push hardware utilization to its limit while maintaining strict tail-latency guarantees.

1.4 Contributions and Scope

This survey provides a systematic synthesis of the state-of-the-art in SLO-aware LLM serving, bridging the gap between hardware-centric optimizations and high-level service management. We contribute:

- A unified taxonomy categorizing scheduling techniques by their operational layer (Engine vs. Cluster) and their primary optimization objective.
- A comparative synthesis of representative frameworks including **SOLA**, **Tempo**, **AdaServe**, and **SCORPIO**, analyzing their trade-offs in TTFT, TPOT, and throughput.
- A practical mapping of these research strategies to the internal architectures of production engines like vLLM [12] and SGLang [25], identifying current implementation gaps.
- A roadmap of open challenges, including the management of heterogeneous GPU clusters and the emerging requirements of multi-modal and agentic workloads.

The following sections detail the technical background (Section 2), our engine-layer taxonomy (Section 3), cluster-layer strategies (Section 4), mapping research to existing LLM frameworks (Section 5) and a concluding discussion on the future of the field (Section 6).

2 Background and Systemic Challenges

The architectural foundations of Large Language Models (LLMs), primarily based on the Transformer architecture, introduce unique systems challenges that differentiate them from traditional deep learning workloads. To understand the necessity of SLO-aware scheduling, one must first analyze the mechanical execution of the inference pipeline and the physical resource constraints of modern GPU hardware.

2.1 Autoregressive Inference: Prefill and Decode Phases

LLM inference is an iterative, autoregressive process where each generated token is appended to the input sequence to predict the subsequent token. This process is bifurcated into two distinct phases, each characterized by orthogonal computational profiles and hardware utilization patterns.

The Prefill Phase(or Encoding phase) occurs at the start of a request. The engine processes the entire input prompt in a single forward pass to compute the initial hidden states. Because the entire prompt is available simultaneously, the GPU can exploit high degrees of parallelism across its arithmetic logic units (ALUs). This phase is typically compute-bound, where performance is limited by the GPU's floating-point operations per second (FLOPS). For schedulers, the prefill phase presents a burst of high-intensity work; while it can be accelerated by larger batch sizes, a single massive prompt can occupy the GPU for a significant duration, stalling all other concurrent tasks.

The Decode Phase(or Generation phase) follows the prefill. Here, the model generates tokens one at a time. Each step requires a full forward pass of the model, but unlike the prefill phase, it only processes a single token which uses the past context stored in the form of a Key-Value(KV) Cache. This makes the decode phase memory-bandwidth bound. The performance is bottlenecked not by the GPU's compute capacity, but by the speed at which model weights and the KV cache can be transferred from High Bandwidth Memory (HBM) to the GPU cores. Consequently, the GPU's compute units often sit idle during decoding, a phenomenon that motivates techniques like continuous batching to fill these compute bubbles with other requests [1, 13].

2.2 Memory Management: The KV Cache Crisis

A central innovation in efficient LLM serving is the caching of intermediate tensors, known as the Key-Value (KV) Cache. To avoid recomputing the attention states of all previous tokens at every decode step, the engine stores the Key and Value vectors for every token in the sequence. While the KV cache dramatically reduces computational redundancy, it introduces a severe memory management crisis.

The KV cache for a single request grows linearly with the number of generated tokens. For massive models with high dimensionality and hundreds of layers, the KV cache can quickly exceed several gigabytes for a single long-running conversation. Traditional memory allocators used in deep learning frameworks often allocate memory in contiguous blocks based on the maximum possible sequence length. This led to *internal fragmentation* (unused space within an allocated block) and *external fragmentation* (unusable gaps between blocks).

The introduction of PagedAttention[12] addressed this by partitioning the KV cache into non-contiguous pages similar to virtual memory in operating systems. While PagedAttention allows for near-zero memory waste from fragmentation, it does not solve the fundamental problem of capacity. A scheduler must still decide which requests to admit based on the available page pool. If the engine admits too many requests that subsequently generate longer-than-expected outputs, the page pool will be exhausted. This forces the system into preemption, either swapping the KV cache to CPU memory or recomputing it later, both of which result in severe SLO violations as latencies spike by orders of magnitude [6, 7].

2.3 Evolution of Batching: From Static to Continuous

Early serving systems used Static Batching, where a group of requests was processed together, and the system waited for the longest request in the batch to finish before starting a new one. This led to massive under-utilization, as the GPU would become increasingly idle as shorter requests finished early (the tail-end problem).

6

Shreyash Kondakindi, Pratistha Gaur, and Umesh Bellur

Continuous Batching(or Iteration-level Batching) revolutionized the field by allowing the engine to insert or remove requests at the granularity of a single iteration. The concept was first formalized in Orca [22], which was then adopted by systems such as vLLM. When a request finishes, its slot in the batch is immediately filled by a new request's prefill. While continuous batching significantly increases throughput, it complicates SLO management. The scheduler now faces a scheduling decision at every single iteration: which requests should be included in the next forward pass to ensure that no individual request's SLO is violated. This transition from static batch boundaries to a fluid, iteration-level queue is the catalyst for advanced research systems such as SCORPIO[19], Tempo[23], and AdaServe[15].

This two-level taxonomy provides a structural lens for understanding where scheduling decisions occur. The engine layer manages fine-grained, iteration-level execution on a single node, while the cluster layer governs request placement and coordination across multiple replicas. Together, they form a hierarchical control plane in which local hardware efficiency and global SLO attainment must be jointly optimized.

3 Engine-Layer Scheduling: Microscopic Resource Arbitration

Engine-layer scheduling represents the inner loop of LLM infrastructure, operating at the granularity of individual GPU execution steps (iterations). Unlike cluster-level routers that handle macro-scale request placement across a fleet of replicas, the engine-layer scheduler is tasked with the precise, iteration-by-iteration management of a single node's internal compute and memory resources, as illustrated in Figure 2. At this level, the primary challenge is the *Resource Contention Paradox*, to remain economically viable, hardware must be maintained at peak saturation (high batch sizes), yet high utilization naturally creates stochastic queuing delays that threaten tail-latency SLOs.

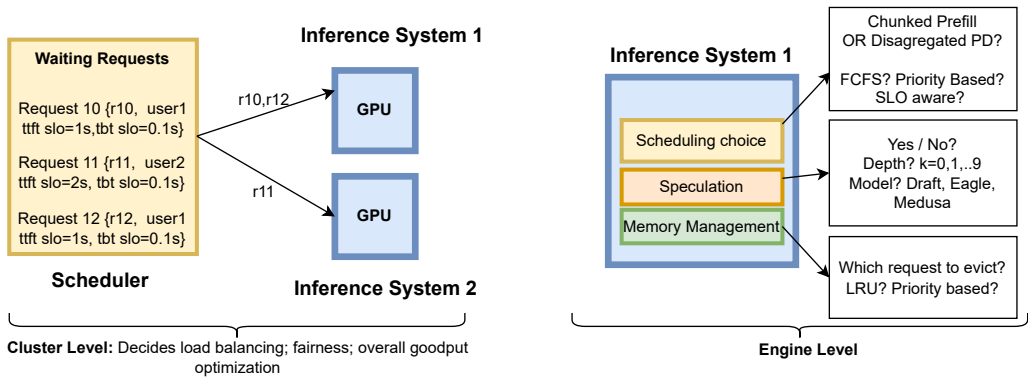


Fig. 2. Overview of cluster-level and engine-level scheduling decisions in LLM inference infrastructure.

3.1 The Physics of the Inference Bottleneck

To understand why engine-layer scheduling is a non-trivial problem, one must analyze the physical constraints of the hardware-software contract in transformer inference.

3.1.1 The Arithmetic Intensity Gap. The primary driver of scheduling complexity is the discrepancy in arithmetic intensity between the two phases of inference.

$$AI = \frac{\text{Operations (FLOPS)}}{\text{Data Movement (Bytes)}} \quad (1)$$

During the Prefill Phase, AI is high because the entire input prompt is processed in parallel, allowing for significant reuse of model weights in the GPU's L1/L2 caches. This phase is compute-bound. Conversely, during the Decode Phase, AI drops by orders of magnitude. Since only one token is generated per pass, the model weights and KV cache must be streamed from HBM for a minimal amount of computation, making it strictly memory-bandwidth bound.

This phase duality means that any batch containing both prefills and decodes will inevitably underutilize one of the two resources (ALUs or HBM). A scheduler's primary job is to shape the batch to minimize these compute bubbles or memory stalls.

3.2 Formalizing the SLO-Aware Scheduling Problem

We can formalize the engine's objective as a constrained optimization problem. Let $\mathcal{R}$ be the set of active requests. For each request $r \in \mathcal{R}$, we define:

- $d_{TTFT}(r)$: The deadline for the first token.
- $d_{TBT}(r)$: The deadline for the interval between tokens i and $i + 1$.
- $C_{pre}(r)$: The compute cost of the prefill.
- $M_{KV}(r, t)$: The KV cache memory required at iteration t .

The scheduler's goal is to maximize Goodput (G):

$$G = \frac{1}{T} \sum_{r \in \mathcal{R}} \mathbb{I}(\text{all deadlines met for } r) \quad (2)$$

subject to:

$$\sum_{r \in \text{Batch}} M_{KV}(r, t) \leq M_{HBM_Total} \quad (3)$$

3.3 Iteration-Level Interleaving and the "Prefill Stall"

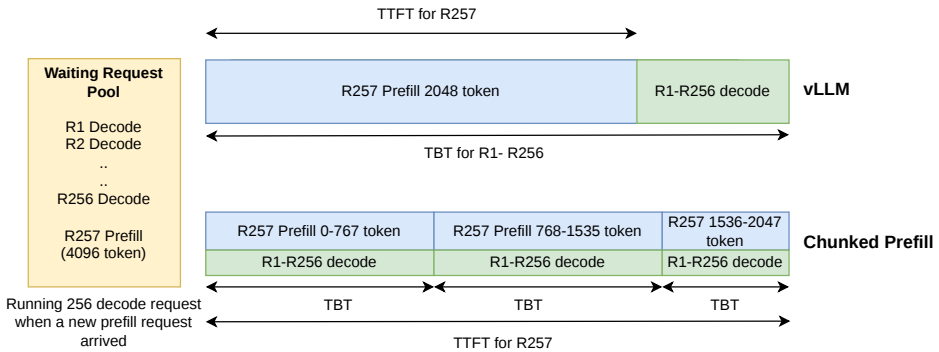


Fig. 3. Illustration of the Prefill Stall. In atomic prefill (top), a 2048-token prefill monopolizes GPU compute, blocking 256 concurrent decode requests and causing severe TBT violations. In chunked prefill (bottom), the prefill is partitioned into smaller segments of 1024 tokens and interleaved with decode batches, preserving streaming smoothness while slightly increasing TTFT for the new request

The introduction of continuous batching [12] enabled the engine to modify the batch composition at every token-generation step. However, this introduced the Prefill Stall problem.

3.3.1 The Impact of Atomic Prefills. In early implementations of vLLM [12], the prefill of a new request was treated as an atomic, non-preemptible operation. For a long prompt (e.g., 4,096 tokens), the GPU commits its full compute resources to processing that prompt for several hundred milliseconds. During this window, all other 256 requests in the decode pool are blocked. As shown in Figure 3, this head-of-line blocking results in severe TBT SLO violations for the active decode requests.

3.3.2 Chunked and Pipelined Prefills. To solve this, Sarathi-Serve[1] and SOLA[7] proposed breaking prefills into chunks of size K (e.g., 1024 tokens) and creating hybrid batches with requests from both phases.

$$T_{iter} \approx \frac{K \cdot \text{Weights}}{BW_{mem}} + \frac{K \cdot \text{Tokens}}{FLOPS} \quad (4)$$

By tuning K , the scheduler can ensure that the total iteration time never exceeds the minimum d_{TBT} of the active pool. This Phase Interleaving ensures that while the new user experiences a slightly higher TTFT, the existing users experience zero jitter in their streaming output. Although selecting a small chunk size K can help meet TBT SLOs, it may lead to underutilization of GPU resources. A small chunk size (e.g., 512 tokens) often fails to fully saturate the GPU's compute resources, resulting in reduced overall throughput.

3.4 Phase Disaggregation

A structural response to this phase duality is Prefill–Decode (PD) disaggregation, which isolates compute-bound prefill workloads from memory-bound decode workloads. This separation is typically implemented at the macro level by routing prefill and decode tasks to entirely different GPU clusters, as demonstrated by DistServe [26] and Splitwise [18]. Alternatively, systems like RAPID-Serve [16] and Semi-PD [8] achieve this disaggregation on a micro level, utilizing fine-grained compute unit partitioning to isolate phases on the same GPU. By decoupling these phases at the architectural level, systems minimize resource contention and maximize the utilization of both ALUs and memory bandwidth. While this section focuses primarily on single-engine scheduling policies, PD disaggregation represents a complementary shift toward system design. The engine-layer optimizations discussed here remain fully applicable to these isolated instances. Furthermore, scaling PD disaggregation to the cluster level opens new avenues for meeting SLOs through dynamic resource allocation; for example, a cluster manager can temporarily provision more GPU nodes to the decode pool to absorb a sudden burst of generation requests.

3.5 Adaptive Speculative Decoding (ASD)

Speculative Decoding is an inference optimization that enables generating multiple tokens per decoding step, in contrast to conventional one-token-at-a-time autoregressive process. By amortizing verification over several tokens, it reduces TPOT and helps meet tighter TPOT SLOs. Speculative Decoding consists of two phases: a **speculation phase** in which several candidate tokens are proposed, and a **verification phase** in which the target model validates these candidates and commits the accepted tokens. Modern speculative decoding mechanisms support multiple forms of speculation. Candidate tokens maybe generated using a lightweight draft model that decodes faster, or via modified output heads within the target model, as in approaches such as Medusa[2] and EAGLE[14], which perform multi-token prediction. In an SLO-aware context, speculative decoding functions not only as a performance optimization but also as a **Priority Escalation Knob**, allowing the system to trade additional compute for faster decoding.

3.5.1 The Speculative Trade-off. AdaServe[15] correctly identifies that speculation is not free. It consumes ALU cycles and memory bandwidth. If the system is under heavy load, speculating on requests that are already meeting their SLOs is wasteful.

To overcome this limitation, recent frameworks have integrated SLO-awareness into speculative decoding, albeit at different operational scopes. AdaServe itself adopts a request-centric approach in which it first bounds the minimum speculation depth (k) needed to meet the target TPOT SLO, and subsequently leverages any remaining compute capacity to expand batching for higher throughput. However, its scheduling remains rigid, as k is fixed throughout the lifespan of the request.

Conversely, AdaSpec implements a dynamic, engine-centric approach. Instead of fixing the speculation depth upfront, AdaSpec continuously recalculates k at each iteration. By factoring in a holistic view of system state, such as fluctuating queue lengths, processing delays, verification costs, and historical acceptance rates—AdaSpec can dynamically tune its speculation strategy to maintain SLOs under variable loads.

BanditSpec[9] uses a multi-armed bandit approach to assign speculative budgets iteration-by-iteration. The authors adopt an online learning strategy that selects among multiple speculation arms in real-time based on a reward signal. Each arm corresponds to a specific speculative configuration, including the choice of draft model, speculation mechanism (e.g., Medusa or EAGLE), and speculation depth. The objective is to identify the hyperparameter configuration that maximizes the number of accepted tokens, thereby minimizing wasted computation. By improving token acceptance efficiency, this approach can indirectly contribute to meeting TPOT SLOs.

Across these methods, the key insight is that both speculation depth k and speculation type serve as primary control knobs, enabling the system to navigate the throughput–latency trade-offs.

- **Urgent Requests:** Assign high speculation depth ($k = 5$ or 8). This pushes the request through the pipeline faster, reducing its queuing and execution time on the GPU.
- **Non-Urgent/Batch Requests:** Assigned $k = 0$ (standard decoding). This saves GPU energy and bandwidth for the urgent requests.

Its important to note that these approaches can help satisfy TPOT SLOs, but they do not guarantee predictable TBT, so, jitter may persist.

3.6 Predictive Memory Management: Taming the KV Cache

As discussed, the KV cache is the dominant scaling bottleneck for long-context LLM serving. For a model with L layers, H heads, head dimension d , and sequence length T , the KV footprint grows as:

$$O(L \cdot H \cdot d \cdot T),$$

making memory consumption linear in generated tokens but multiplied across the full stack depth. Unlike model weights (static) or activations (ephemeral), KV memory is persistent for the lifetime of a request. This persistence turns GPU HBM into a stateful resource whose occupancy increases monotonically with generation length. PagedAttention [12] mitigates internal fragmentation by virtualizing the KV cache and paging at block granularity. However, it does not change the fundamental capacity constraint: once physical HBM is exhausted, the system must evict KV blocks to host memory. Thus, the core problem is not allocation efficiency, but admission under uncertain future growth.

3.6.1 The Preemption Death Spiral. When HBM utilization reaches 100%, the scheduler is forced into reactive eviction. A victim request's KV blocks are swapped to CPU memory over PCIe (or NVLink), introducing a latency cost:

$$T_{\text{swap}} = \frac{\text{KV size}}{\text{interconnect bandwidth}}.$$

10

Shreyash Kondakindi, Pratistha Gaur, and Umesh Bellur

For long generations, the KV size can reach multiple gigabytes, making T_{swap} or Swap-In latency comparable to or larger than the original prefill latency. This leads to a cascading failure where the system spends 80% of its time moving data over the PCIe bus and only 20% on inference.

3.6.2 Past-Future and Probabilistic Admission. The **Past-Future Scheduler** [6] uses a neural predictor to estimate the *Total Generation Length* of a request upon arrival. By calculating the memory trajectory of the current batch, the scheduler can see that a collision is likely in the next N iterations. It then proactively delays the admission of new prefills, keeping the HBM usage at a steady 95% without ever hitting the 100% preemption trigger.

This proactive methodology addresses one of the most severe failure modes in long-context inference: the *preemption death spiral*. In traditional reactive systems, when physical High Bandwidth Memory (HBM) reaches 100% utilization, the scheduler is forced to evict active Key-Value (KV) blocks to host memory over the PCIe bus. For long-context generations, these KV caches can reach multiple gigabytes, making the subsequent swap-in latency comparable to or even larger than the original prefill latency. This results in a cascading failure where the system may spend up to 80% of its time moving data rather than performing inference.

By utilizing probabilistic admission control, the Past-Future scheduler shifts the system's focus from raw utilization to latency stability. The analysis reveals a critical strategic trade-off of Sacrifice Vs Protection where this delay acts as a buffer to protect the TBT latency of in-flight requests, preventing the jitter in the output stream that occurs when decode iterations are starved or preempted.

Ultimately, this approach highlights a broader architectural transition identified in the paper. Admitting an LLM request is no longer viewed as a simple task execution, but as a long-term, stateful commitment to providing increasing memory capacity across subsequent iterations. By maintaining HBM at a steady 95%, the Past-Future scheduler treats the engine as a feedback-driven control plane that balances the stochastic nature of autoregressive growth against hard hardware limits.

3.6.3 Predictive Prefetching. While predictive admission control avoids crossing the memory boundary, an orthogonal strategy is to expand the effective boundary itself. Pie [20] observes that LLM decoding exhibits highly predictable memory access patterns across layers. At each decoding step, the model consumes KV blocks layer-by-layer in a fixed order. This determinism enables prefetching. Unlike traditional swapping where a block is moved only after a page fault or eviction event, Pie treats CPU memory as an elastic extension of GPU HBM. Instead of keeping all KV blocks resident in GPU memory, it maintains a working set on HBM and places colder blocks in CPU memory. Crucially, blocks are prefetched back to GPU memory before they are accessed in the forward pass.

3.7 Saturation Point and Throughput-Latency Analysis

A critical component of engine-layer scheduling is identifying the Saturation Point, which marks the threshold where incremental throughput gains no longer justify the corresponding latency penalties. This relationship is defined by the hardware constraints of the GPU memory subsystem during the decode phase. As the batch size B increases, Throughput T scales according to:

$$T \propto \frac{B}{T_{\text{base}} + \alpha B} \quad (5)$$

In this model, T_{base} represents the fixed overhead of streaming model weights from High Bandwidth Memory (HBM) to the GPU registers, a cost that is amortized across the batch. Conversely, α

represents the incremental memory bandwidth required to read and write the KV cache for each individual request. While throughput eventually plateaus toward a limit of $1/\alpha$, the Latency L continues to grow linearly:

$$L \approx T_{base} + \alpha B \quad (6)$$

To maintain system stability, SLO-aware schedulers must continuously solve for the maximum batch size that satisfies the target latency:

$$\max B \quad \text{s.t.} \quad L(B) \leq \text{SLO}_{Target} \quad (7)$$

This dynamic batch sizing acts as a critical safeguard against widespread SLO violations. During sudden request bursts, rather than maximizing hardware utilization, these schedulers may proactively shrink the batch size. By reducing the per-iteration delay, the engine can flush the queue and clear urgent requests that would otherwise be caught in a high-latency convoy effect. Once the burst subsides and the queue becomes manageable, the engine transitions back to high-throughput operation by increasing the batch size once more.

However, real LLM workloads are heterogeneous. Requests differ in prompt length, predicted generation length, decode intensity, and urgency. Two batches of identical size B can produce dramatically different tail latencies depending on which requests are grouped together. Recent work such as Efficient LLM Scheduling by Learning to Rank [5] frames scheduling as a ranking problem. Instead of heuristically ordering requests (e.g., FCFS or shortest-first), the scheduler learns a scoring function that predicts which requests should be admitted or prioritized to optimize a global objective (e.g., throughput under SLO constraints). Learning-to-rank schedulers replace fixed heuristics with a learned scoring function that prioritizes requests based on features such as prompt length, predicted generation length, waiting time, and SLO slack. At each step, the top- B ranked requests are admitted into the batch. Trained on workload traces to minimize latency or SLO violations, the learned policy intelligently mixes short and long requests, reducing head-of-line blocking and improving tail latency without sacrificing throughput.

3.8 Comparison of Architectural Innovations

To synthesize the engine-layer landscape, we compare the foundational systems in Table 1. The evolution of these architectures reflects a fundamental shift from hardware-centric utilization to service-centric control.

Early frameworks, spearheaded by **vLLM** [12], focused on resolving the physical memory constraints of the Transformer architecture. By introducing *PagedAttention*, vLLM decoupled logical KV cache sequences from physical HBM blocks, effectively eliminating external fragmentation. However, while this approach maximized raw throughput through continuous batching, it remained largely phase-agnostic. In vLLM-style schedulers, a long incoming prefill can monopolize the GPU's ALUs for hundreds of milliseconds, creating a prefill stall that blocks the execution of dozens of concurrent decode requests. This results in high jitter and significant violations of TBT SLOs.

Subsequent innovations focused on mitigating this temporal interference. **Sarathi-Serve** [1] introduced chunked prefills and hybrid batching to smooth the execution profile. By partitioning a single large prefill into smaller, predictable compute chunks and interleaving them with decode iterations, Sarathi-Serve stabilizes the TBT curve. This represents the first major shift toward "phase-aware" scheduling, where the system intentionally trades a slight increase in Time to First Token (TTFT) for the sake of generation smoothness and latency predictability.

As models grew and context windows expanded, the bottleneck shifted from compute-bound stalls to memory-induced instability. The **Past-Future Scheduler** [6] identifies that reactive memory management—waiting for a HBM trigger before preempting requests—leads to a preemption death spiral. When multiple requests are evicted to the CPU over the relatively slow PCIe bus, the resulting

Table 1. Comparison of Engine-Layer Scheduling Architectures

System	Primitive	Granularity	Phase-aware	KV-aware	Primary outcome
vLLM [12]	PagedAttention + cont. batching	Iter.	Limited	✓	High throughput via efficient KV paging and large feasible batches.
Sarathi-Serve [1]	Chunked prefills (hybrid batches)	Chunk+iter.	✓	Implicit	Reduce prefill stalls; stabilize inter-token latency under bursts.
AdaServe [15]	Adaptive speculative decoding	Iter.	Indirect	–	Lower tail latency by allocating speculation depth to urgent requests.
Past-Future [6]	Predictive admission control	Admission+iter.	–	✓	Avoid KV-induced preemption via generation-length / memory forecasting.
Pie [20]	Memory management / capacity expansion	Iter.	–	✓	Expand effective KV memory capacity with negligible latency impact.
SOLA [7]	Slack-/state-aware batching	Iter.	✓	✓	Maximize goodput subject to per-request SLO deadlines.
Learning to Rank [5]	Request prioritization	Request+Iter.	–	–	Length-aware request prioritization to mitigate head-of-line blocking and improve latency.

swap-in latency can exceed the original prefill time, causing a cascading system failure. Past-Future shifts this to a predictive paradigm: by forecasting the memory trajectory of a batch using neural predictors, it implements probabilistic admission control to keep HBM usage steady, preventing costly preemptions entirely.

The most recent generation of schedulers, such as **AdaServe** [15] and **SOLA** [7], introduces the concept of Priority Escalation and Slack-Awareness. Slack is the time remaining before a deadline. AdaServe re-imagines speculative decoding not just as a performance optimization, but as a scheduling primitive. It dynamically allocates speculation depth to requests nearing their SLO deadlines, essentially spending extra compute cycles to buy back time for urgent tasks. Similarly, SOLA focuses on maximizing Goodput: the subset of throughput that strictly adheres to SLO targets—by using slack to determine batching priorities.

Collectively, these systems illustrate that phase-awareness, KV-memory modeling, and deadline sensitivity are orthogonal but complementary dimensions of the same problem. This suggests that the engine layer is transitioning from a simple resource arbitration mechanism (like FCFS or Round-Robin) to a sophisticated, feedback-driven control plane. In this new paradigm, the scheduler must continuously balance three competing tensions:

- (1) Utilization: Keeping the GPU ALUs and memory bandwidth saturated.
- (2) Stability: Avoiding memory-induced preemption and the "death spiral."
- (3) Responsiveness: Meeting multi-dimensional SLOs (TTFT, TBT, and TTLT) in the face of bursty and heterogeneous workloads.

In the next section, we broaden this perspective to the cluster layer, where routing, admission control, and multi-tenant coordination introduce additional degrees of freedom and complexity to this optimization problem.

4 Cluster-Layer and Cross-Layer Scheduling

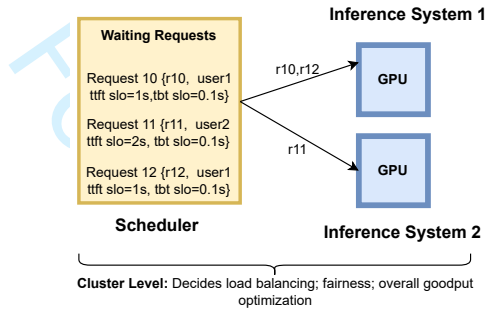


Fig. 4. Overview of cluster-level decisions in LLM inference infrastructure

Cluster layer scheduling and request routing govern the distribution of tasks across multiple inference engines or GPU replicas as shown in Figure 4. While engine layer scheduling manages the hardware at the level of individual token iterations, the cluster layer operates as a global control plane. It manages the global request queue, admission control, and load balancing to optimize for aggregate attainment of service level objectives across thousands of concurrent users.

4.1 Deadline Aware Routing and Request Partitioning

A primary challenge in cluster level management is workload heterogeneity. Systems such as Tempo[23] and SLOs-Serve[3] observe that treating all requests as a single homogeneous pool leads to head of line blocking. This occurs when a long running batch request such as a document summarization task stalls a latency sensitive interactive request such as a real time chat message.

To mitigate this, cluster layer schedulers use application aware routing. Tempo categorizes traffic into distinct patterns including interactive streaming which prioritizes time to first token and time between tokens as well as batch tools which prioritize time to last token. The scheduler dynamically partitions cluster resources based on these specific needs. By using deadline aware queues, the router can estimate the expected waiting time at each engine replica. It routes requests to the node most likely to meet the specific deadline of that request. This prevents the over provisioning typical of static round robin routing and ensures that high priority interactive requests are not queued behind non urgent batch work.

4.2 Credit Based Fairness and Global Coordination

In multi tenant environments, maximizing global throughput often comes at the cost of fairness. SCORPIO[19] introduces a credit based batching mechanism to address this at the cluster level. Instead of each engine acting as an isolated black box, SCORPIO coordinates across nodes to ensure that no single tenant or noisy neighbor monopolizes the compute capacity of the cluster.

14

Shreyash Kondakindi, Pratistha Gaur, and Umesh Bellur

The system tracks credits representing normalized compute time across the cluster. The gateway can throttle or prioritize requests based on these credit balances. If a user issues a burst of long prompt requests that would exhaust the global memory pool, the credit based governor slows their admission. This protects the latency of other tenants and ensures that the system maintains fair attainment. This metric balances the percentage of met objectives across all active users regardless of their workload intensity.

4.3 Cross Layer Co Design and Predictive Modeling

The most recent advancement in the field is the shift toward cross layer scheduling where the cluster router and the engine layer scheduler share a unified performance model. This approach moves away from the black box abstraction where the router knows nothing about engine internals. Systems such as SynergySched[11] represent this holistic approach. Rather than the router making decisions based on simple heuristics like queue length, it uses a predictive performance model to forecast how adding a specific request to a specific engine will impact the latencies of already running requests.

This coordination includes engine to cluster feedback where the engine exports its internal state such as KV cache occupancy and current batch latency to the router. The cluster layer can then dynamically tune engine layer knobs such as the maximum token budget or the speculative decoding depth to react to global workload shifts. This synergy allows systems to push hardware utilization closer to the saturation point without the catastrophic tail latency spikes that occur in decoupled systems.

Heterogeneous requirements do not always imply a multi tenant deployment. A single application may internally issue requests with different latency priorities. However, in practice, these distinct objectives most commonly arise in multi tenant environments where different users or services share the same infrastructure and impose unique performance targets.

Takeaway: The Role of the Cluster Router

Mechanism: Deadline aware routing and credit based admission.

Goal: Prevent head of line blocking and ensure multi tenant fairness.

Key Innovation: Moving from hardware agnostic load balancing to SLO aware placement that considers the real time KV cache load and predicted performance of each replica.

4.4 Cross-Layer Co-Design: The Predictive Frontier

The most recent advancement in the field is the shift toward *Cross-Layer* scheduling, where the cluster router and the engine-layer scheduler share a unified performance model. This approach moves away from the blackbox abstraction where the router knows nothing about engine internals.

Systems such as SynergySched[11] represent this holistic approach. Rather than the router making decisions based on simple heuristics like queue length, it uses a predictive performance model (e.g., the LENS and PRISM frameworks) to forecast how adding a specific request to a specific engine will impact the latencies of already-running requests.

- **Engine-to-Cluster Feedback:** The engine exports its internal state, such as KV cache occupancy and current batch latency, to the router.
- **Dynamic Configuration:** The cluster layer can dynamically tune engine-layer knobs, such as the maximum token budget or the speculative decoding depth, to react to global workload shifts.

This synergy allows systems to push hardware utilization closer to the saturation point without the catastrophic tail-latency spikes that occur in decoupled systems.

4.5 Comparison of Cluster-Level Strategies

Table 2 highlights the diversity of control strategies employed at the cluster layer and underscores how these systems differ in both decision scope and SLO handling philosophy. Unlike engine-layer schedulers that operate at the level of token iteration and GPU utilization, cluster-layer systems primarily act through routing, admission control, and placement decisions across replicas. Tempo and SLOs-Serve focus on deadline-aware routing and prioritization, using queue state and request urgency to improve the likelihood of meeting latency targets. SCORPIO extends this view by incorporating fairness-driven admission control, explicitly addressing multi-tenant interference under heterogeneous SLOs. SynergySched moves a step further by introducing cross-layer coordination, allowing routing decisions to incorporate engine-level signals such as load or predicted service time. Helix represents a distinct design point, framing placement itself as an optimization problem over heterogeneous hardware. Taken together, the table shows a progression from reactive load distribution toward predictive, policy-driven coordination, where the cluster layer acts as a global control mechanism shaping system-wide SLO attainment.

Table 2. Comparison of Cluster-Layer Scheduling Systems

System	Core Idea	Decision Scope	SLO Handling	Primary outcome
Tempo [23]	Deadline-aware routing	Replica selection	TTFT, TTLT	Route requests to replicas most likely to meet deadlines.
SLOs-Serve [3]	Multi-queue prioritization	Admission + routing	Mixed SLO classes	Improve SLO attainment by prioritizing urgent requests.
SCORPIO [19]	Credit-based control	Admission control	Fairness + latency	Prevent starvation across tenants with different SLOs.
SynergySched [11]	Predictive two-layer control	Routing + coordination	General latency SLOs	Stabilize system performance via cross-layer feedback.
Helix [17]	Optimization-based placement	Model + replica placement	Latency + throughput	Map workloads across heterogeneous GPUs efficiently.

5 Mapping Research to Practice: vLLM and SGLang

While the research systems discussed in Sections 3 and 4 offer specialized SLO-aware mechanisms, production environments primarily rely on general-purpose open-source engines. We now map academic strategies to the configuration knobs and runtime behaviors of the two most prominent frameworks: vLLM and SGLang.

5.1 vLLM: Throughput-Oriented Foundation

vLLM is the industry standard for high-throughput LLM serving, built primarily around PagedAttention[12]. From an SLO perspective, vLLM provides the foundational mechanisms but leaves the policy to the user.

- **Continuous Batching Control:** vLLM exposes the max_num_batched_tokens parameter. In research terms, this is the token budget. Tuning this knob allows operators to trade off

throughput for latency; a lower budget reduces the compute time per iteration (improving TBT) at the expense of lower hardware utilization.

- **Prefill-Decode Balancing:** Early descriptions of vLLM emphasized prioritizing prefills to optimize TTFT. Newer versions enable *chunked prefill* using a knob `enable_chunked_prefill` as discussed in section 3.3, breaking large prompts into smaller token segments that can be interleaved with ongoing decode iterations. Importantly, current vLLM knobs expose only *static* chunk sizing policies, recent research prototypes have modified the engine to support *dynamic chunk sizing*, where prefill chunk size is adjusted at runtime based on queue pressure, SLO slack, or predicted iteration time.
- **Concurrency Caps as Tail-Latency/Fairness Stabilizers:** While `max_num_batched_tokens` budgets work per iteration, knobs like `max_num_seqs` cap the number of concurrent sequences competing for that budget, shaping queueing delay and tail behavior. In heterogeneous-SLO settings, these caps act as coarse fairness and tail-latency stabilizers, but they are not SLO-aware by themselves.
- **Reactive Admission:** vLLM handles memory pressure reactively through request preemption (recompute or swap). Practitioners can approximate predictive admission control by setting conservative `gpu_memory_utilization` thresholds, though this lacks the prompt-length-aware intelligence discussed in Section 3.6

5.2 SGLang: Structured Serving and Prefix Caching

SGLang extends the capabilities of engines like vLLM by introducing a structured programming interface and a RadixAttention manager [25].

- **Latency Gains via Prefix Caching:** For multi-turn conversations or few-shot prompting, SGLang’s automatic prefix caching drastically reduces the prefill work for repeated tokens. From an SLO perspective, this buys back TTFT budget, allowing the scheduler more room to prioritize decodes for other requests.
- **Structured Parallelism and TTLT Reduction:** SGLang allows complex agentic workflows to be expressed as structured programs with multiple independent branches. These branches can execute concurrently within a single request, enabling intra-request parallelism. By overlapping independent generation steps, SGLang can reduce the overall Time to Last Token (TTLT), even if per-token latency remains unchanged.

5.3 Gaps: Open-Source vs. Research Systems

Despite their efficiency, a significant gap exists between these production engines and the SLO-aware research state-of-the-art.

Table 3. Feature Gaps: Production Engines vs. SLO Research

Feature	Production (vLLM/SGLang)	Research (SOLA/Tempo/AdaServe)
SLO Specification	None (Manual tuning)	Per-request TTFT/TPOT targets
Phase Balancing	Fixed (Prefill-priority Or Chunked Prefill)	Dynamic phase-interleaving
Speculation	Static depth	Adaptive per-request depth
Cluster Routing	Round-robin / Least-request	Deadline-aware / Credit-based

The primary missing link is First-Class SLO Semantics. Current engines treat all requests as equal. To reach the performance levels of systems like SCORPIO or SynergySched, open source frameworks would need to move from best effort iteration scheduling to deadline driven iteration

scheduling, where the hardware forward-pass is determined by the relative urgency of the requests in the active set.

Table 4. Taxonomy of SLO-Aware LLM Scheduling Systems

System	Level	Key Mechanism	Target SLOs	Key Advantage
vLLM	Engine	PagedAttention	Throughput	Efficient memory utilization
AdaServe	Engine	Speculative Decoding	TPOT	Adaptive speculation depth
SOLA	Engine	State-aware sched.	TTFT/TPOT	Dynamic prefill/decode balance
Tempo	Cluster	Priority Queuing	Mixed (TTFT/TTLT)	Deadline-aware routing
SCORPIO	Cross-layer	Credit-based batching	Fairness/TPOT	Cross-node fair allocation
SynergySched	Cross-layer	Predictive modeling	General SLOs	Proactive resource planning

6 Discussion: Trade-offs and Open Challenges

The landscape of LLM scheduling is defined by a series of fundamental trade-offs. As systems move from best-effort throughput to strict SLO attainment, the complexity of the control plane increases significantly. We discuss these tensions and identify the remaining hurdles for the next generation of serving systems.

6.1 Key Trade-offs

Throughput vs. Latency (The Goodput Frontier). Almost all surveyed techniques grapple with the inverse relationship between hardware utilization and tail latency. While continuous batching improves throughput, it introduces iteration jitter as new prefills preempt running decodes. Systems like SOLA and SynergySched demonstrated that the Pareto frontier can be pushed outward by using phase-aware scheduling, but there is no free lunch - achieving a 99th-percentile TTFT SLO typically requires running at a lower aggregate throughput to maintain a headroom for bursty arrivals.

Predictive Accuracy vs. System Overhead. The shift toward predictive scheduling (e.g., Past-Future, SynergySched) introduces a new overhead: the cost of the performance model itself. While predicting request length can prevent costly KV cache evictions, the model must be lightweight enough to run at line-rate. Furthermore, if a prediction is wrong, the system may either under-utilize resources (false positive) or trigger the very OOM it sought to avoid (false negative). Developing robust uncertainty-aware schedulers remains a critical need.

The Predictability Gap

Current systems assume relatively stable request distributions. A major challenge is maintaining SLOs during **distribution shifts** (e.g., a sudden change in average prompt complexity or output length), where the scheduler’s internal models may become stale.

6.2 Open Challenges and Future Directions

Heterogeneous Hardware Scheduling: Existing scheduling research largely assumes homogeneous GPU clusters, where all replicas exhibit similar compute and memory characteristics. In contrast, recent systems research has begun to explicitly consider heterogeneous accelerator environments. For example, Helix formulates LLM inference serving as an optimization problem over heterogeneous GPU clusters, jointly deciding model placement and request scheduling across devices

with differing capabilities [17]. Similarly, prior work on SLA-driven inference and GPU cluster schedulers has explored device-aware resource management across mixed accelerators [4, 21].

Despite these advances, most LLM serving schedulers—including chunked-prefill and state-aware systems such as SOLA—are evaluated under homogeneous hardware assumptions and do not explicitly model device-level performance variability. This creates a gap between research prototypes and real-world deployments.

Scheduling for SLO attainment across heterogeneous devices therefore remains an open challenge. Performance characteristics vary substantially across GPU generations, particularly in the ratio of compute throughput to memory bandwidth, which directly affects prefill–decode balance and KV-cache dynamics. A cross-layer scheduler must decide not only *when* to run a request, but also *where* to place it—i.e., which class of hardware is most likely to satisfy the request’s latency target.

Multi-Tenant Isolation and Fairness. While **SCORPIO** introduces credit-based fairness, true performance isolation in LLM serving is still in its infancy. In a multi-tenant cloud, one noisy neighbor issuing extremely long prompts can exhaust the global KV cache pool. Future research is needed in hardware-enforced isolation and spatial partitioning of the KV cache, ensuring that a surge in one tenant’s traffic does not degrade the TBT of another.

Autonomic Serving and RL-based Control. As the number of control knobs in engines such as vLLM grows (speculation depth, token budget, block size), manual tuning becomes impossible. There is a growing research interest in applying Reinforcement Learning (RL) to LLM scheduling. An autonomic scheduler could learn the optimal batching and routing policies in real-time by observing the reward signal of SLO attainment, potentially outperforming the heuristic-based policies currently used in Tempo or vLLM. Note that currently, many of these parameters are configured statically; a promising direction is to make them dynamically adjustable at runtime through closed-loop control or reinforcement learning, allowing the system to adapt continuously to workload shifts.

Beyond Text: Multi-Modal SLOs. As LLMs evolve into Multi-Modal Models (LMMs) that process video and audio, the scheduling constraints will shift again. Processing an image has a fixed prefill cost, but streaming a video requires continuous high-bandwidth state management. Defining and meeting SLOs for multi-modal streams—where the token is no longer the atomic unit of latency—is a nascent and vital research direction.

7 Conclusion

The rapid proliferation of Large Language Models has transformed LLM serving from a best-effort throughput challenge into a complex multi-objective optimization problem. This survey has organized the landscape of SLO-aware scheduling into a two-layer taxonomy: engine-layer optimizations that manage the micro-level interleaving of prefill and decode phases, and cluster-layer frameworks that coordinate multi-node resources to maintain global fairness and deadline compliance.

Our synthesis reveals that the future of efficient LLM serving lies in *fine-grained adaptivity*. As demonstrated by systems like AdaServe and SOLA, static scheduling policies—such as fixed speculative depths or prefill-priority batching—are increasingly insufficient for heterogeneous workloads. Instead, the next generation of serving engines must treat scheduling parameters as dynamic knobs, adjusted in real-time based on per-request urgency and predictive performance models.

Furthermore, bridging the gap between research-level SLO awareness and production frameworks like vLLM and SGLang remains a critical frontier. While these engines provide the foundational

mechanisms for high-throughput serving, they lack the first-class SLO semantics necessary to provide performance isolation in multi-tenant environments. By integrating the predictive admission control and cross-layer coordination strategies discussed in this survey, practitioners can move beyond reactive preemption toward proactive, reliable, and SLO-guaranteed LLM services.

Ultimately, as LLMs evolve toward multi-modal capabilities and agentic workflows, the definition of service quality will continue to expand. The scheduling principles established here—centered on phase awareness, state sensitivity, and predictive coordination—will serve as the architectural bedrock for the scalable AI infrastructure of the future.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA, 117–134.
- [2] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. *arXiv preprint* (2024). <https://arxiv.org/abs/2401.10774>
- [3] S. Chen et al. 2024. SLOs-Serve: Multi-SLO Optimized Scheduling for LLM Inference Serving. *arXiv preprint* (2024).
- [4] Jaehyun Cho et al. 2022. SLA-Driven Machine Learning Inference Framework for Clouds with Heterogeneous GPUs. In *MLSys*.
- [5] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. In *Advances in Neural Information Processing Systems*.
- [6] R. Gong et al. 2025. Past-Future Scheduler for LLM Serving under SLA Guarantees. In *Proceedings of ASPLOS 2025*. 798–813.
- [7] K. Hong et al. 2025. SOLA: Optimizing SLO Attainment for Large Language Model Serving with State-Aware Scheduling. In *Proceedings of MLSys 2025*. To appear.
- [8] Ke Hong, Lufang Chen, Zhong Wang, Xiuhong Li, Qiuli Mao, Jianping Ma, Chao Xiong, Guanyu Wu, Buhe Han, Guohao Dai, Yun Liang, and Yu Wang. 2025. semi-PD: Towards Efficient LLM Serving via Phase-Wise Disaggregated Computation and Unified Storage. *arXiv:2504.19867 [cs.CL]* <https://arxiv.org/abs/2504.19867>
- [9] Yunlong Hou, Fengzhuo Zhang, Cunxiao Du, Xuan Zhang, Jiachun Pan, Tianyu Pang, Chao Du, Vincent Y. F. Tan, and Zhuoran Yang. 2025. BanditSpec: Adaptive Speculative Decoding via Bandit Algorithms. *arXiv:2505.15141 [cs.LG]* <https://arxiv.org/abs/2505.15141>
- [10] J. Huang et al. 2025. SLO-Aware Scheduling for Large Language Model Inferences. *arXiv preprint arXiv:2504.14966* (apr 2025).
- [11] X. Jiang et al. 2025. SynergySched: A Predictive and Synergistic Two-Layer Scheduling Framework for LLM Serving. *arXiv preprint arXiv:2509.23384* (sep 2025).
- [12] W. Kwon et al. 2023. vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention. *arXiv preprint arXiv:2307.10169* (2023).
- [13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joey Gonzalez, Hao Zhang, and Ion Stoica. 2023. vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention. <https://blog.vllm.ai/2023/06/20/vllm.html>.
- [14] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. In *NeurIPS 2025*. <https://arxiv.org/abs/2503.01840>
- [15] Z. Li et al. 2025. AdaServe: SLO-Customized LLM Serving with Fine-Grained Speculative Decoding. *arXiv preprint arXiv:2501.12162* (jan 2025).
- [16] Amna Masood, Pratishtha Gaur, and Nuwan Jayasena. 2026. RAPID-Serve: Resource-efficient and Accelerated P/D Intra-GPU Disaggregation. *arXiv:2601.11822 [cs.DC]* <https://arxiv.org/abs/2601.11822>
- [17] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. 2024. Helix: Distributed Serving of Large Language Models via Max-Flow on Heterogeneous GPUs. *arXiv preprint arXiv:2406.01566* (2024).
- [18] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. 118–132. doi:10.1109/ISCA59077.2024.00019
- [19] Y. Tang et al. 2025. SCORPIO: Serving the Right Requests at the Right Time for Heterogeneous SLOs in LLM Inference. *arXiv preprint arXiv:2505.23022* (may 2025).
- [20] Yi Xu, Ziming Mao, Xiangxi Mo, Shu Liu, and Ion Stoica. 2024. Pie: Pooling CPU Memory for LLM Inference. *arXiv:2411.09317 [cs.LG]* <https://arxiv.org/abs/2411.09317>
- [21] Zhen Ye et al. 2024. Deep Learning Workload Scheduling in GPU Datacenters. *ACM Computing Surveys / ACM* (2024).

[22] {Gyeong In} Yu, {Joo Seong} Jeong, {Geon Woo} Kim, Soojeong Kim, and {Byung Gon} Chun. 2022. ORCA: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022 (Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022)*. USENIX Association, 521–538. Publisher Copyright: © 2022 by The USENIX Association. All rights reserved.; 16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022 ; Conference date: 11-07-2022 Through 13-07-2022.

[23] W. Zhang et al. 2025. Tempo: Application-aware LLM Serving with Mixed SLO Requirements. *arXiv preprint arXiv:2504.20068* (apr 2025).

[24] Ranran Zhen, Juntao Li, Yixin Ji, Zhenlin Yang, Tong Liu, Qingrong Xia, Xinyu Duan, Zhefeng Wang, Baoxing Huai, and Min Zhang. 2024. Taming the Titans: A Survey of Efficient LLM Inference Serving. *arXiv preprint* (2024).

[25] L. Zheng et al. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*. To appear; arXiv:2312.07872.

[26] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>

March 02, 2026

To whom it may concern,

Please find our survey paper titled "LLM Serving under SLOs: A Survey of Engine and Cluster-Layer Scheduling" submitted for consideration for publication in ACM Computing Surveys. We confirm that this manuscript is original work and has not been published previously, in whole or in part. We also confirm that this manuscript is currently not under review by any other journal, conference, or publication venue.

This survey is well suited for ACM Computing Surveys because it systematically synthesizes the emerging body of research on SLO-aware scheduling and resource management for large language model inference. With the rapid growth of LLM deployment in latency-sensitive and multi-tenant environments, a consolidated analysis of system architectures, scheduling strategies, and open research challenges is both timely and directly aligned with the journal's scope.

Thank you for your consideration.

Sincerely,
Pratishtha Gaur
p1gaur@ucsd.edu
On behalf of all authors